Lists

Java Lists

A ***list*** is a common ADT (ABSTRACT DATA TYPE) for holding ordered data.

Operation	Description	Example starting with list: 99, 77
Append(list, x)	Inserts x at end of list	Append(list, 44), list: 99, 77, 44
Prepend(list, x)	Inserts x at start of list	Prepend(list, 44), list: 44, 99, 77
InsertAfter(list, w, x)	Inserts x after w	InsertAfter(list, 99, 44), list: 99, 44, 77
Remove(list, x)	Removes x	Remove(list, 77), list: 99
Search(list, x)	Returns item if found, else returns null	Search(list, 99), returns item 99 Search(list, 22), returns null
Print(list)	Prints list's items in order	Print(list) outputs: 99, 77
PrintReverse(list)	Prints list's items in reverse order	PrintReverse(list) outputs: 77, 99
Sort(list)	Sorts the lists items in ascending order	list becomes: 77, 99
IsEmpty(list)	Returns true if list has no items	For list 99, 77, IsEmpty(list) returns false
GetLength(list)	Returns the number of items in the list	GetLength(list) returns 2

Java ArrayList

The ArrayList class is a resizable array, which can be found in the java.util package.

The difference between a built-in array and an ArrayList in Java, is that the size of an array cannot be modified (if you want to add or remove elements to/from an array, you have to create a new one). While elements can be added and removed from an ArrayList whenever you want. The syntax is also slightly different:

```
import java.util.ArrayList; // import the ArrayList class

ArrayList<String> cars = new ArrayList<String>(); // Create an ArrayList object
```

ArrayList Methods

add()	`add(element)`
	Create space for and add the element at the end of the list.
		`// List originally empty`
		`valsList.add(31); // List now: 31`
		`valsList.add(41); // List now: 31 41`

get()	`get(index)`
	Returns the element at the specified list location known as the ***index***.
	Indices start at 0.
		`// List originally: 31 41 59. Assume x is an int.`
		`x = valsList.get(0);  // Assigns 31 to x`
		`x = valsList.get(1);  // Assigns 41`
		`x = valsList.get(2);  // Assigns 59`
		`x = valsList.get(3);  // Error: No such element`

set()	`set(index, element)` Replaces the element at the specified position in this list with the specified element.	`// List originally: 31 41 59` `valsList.set(1, 119);  // List now 31 119 59`
size()	`size()` Returns the number of list elements.	`// List originally: 31 41 59. Assume x is an int.` `x = valsList.size();  // Assigns x with 3`

ArrayList Capacity

ensureCapacity() method is used to increase the current capacity of an ArrayList. However, capacity of an ArrayList is automatically increased when we try to add more elements than the current capacity.

```
ArrayList<String> list = new ArrayList<String>();

//Now 'list' can hold 10 elements (Default Initial Capacity)

list.ensureCapacity(20);

//Now 'list' can hold 20 elements.
```

trimToSize() method is used to trim the capacity of arrayList to the current size of ArrayList

```
list.add("ONE");

list.add("TWO");

list.add("THREE");

list.add("FOUR");

//reducing the current capacity to current size of an ArrayList.

list.trimToSize();
```

ArrayList Other Methods

isEmpty() method of ArrayList is used to check whether the given ArrayList is empty

```
System.out.println(list.isEmpty());    //Output : true or false
```

contains() method of ArrayList, we can examine whether the ArrayList contains the given element or not.

```
System.out.println(list.contains("ONE"));    //Output : true or false
```

indexOf() and **lastIndexOf()** methods to find out the position of a given element in an ArrayList

```
System.out.println(list.indexOf("ONE"));    //Output : An integer – index of where ONE
                                                is present
```

ArrayList Other Methods

toArray() method returns an array containing all elements of the ArrayList

```
ArrayList<Integer> list = new ArrayList<Integer>();
Object[] array = list.toArray();
```

subList() method returns a view of a portion of an ArrayList in the given range.

```
ArrayList<Integer> list = new ArrayList<Integer>();
List<Integer> subList = list.subList(1, 4);
```

use **addAll()** method which takes Collection type as an argument to join two ArrayLists

Linked List

Linked List is identical to ArrayList.

How the ArrayList works

The `ArrayList` class has a regular array inside it. When an element is added, it is placed into the array. If the array is not big enough, a new, larger array is created to replace the old one and the old one is removed.

How the LinkedList works

The `LinkedList` stores its items in "containers." The list has a link to the first container and each container has a link to the next container in the list. To add an element to the list, the element is placed into a new container and that container is linked to one of the other containers in the list.

When To Use

Use an `ArrayList` for storing and accessing data, and `LinkedList` to manipulate data.

Linked List Methods

Method	Description
addFirst()	Adds an item to the beginning of the list.
addLast()	Add an item to the end of the list
removeFirst()	Remove an item from the beginning of the list.
removeLast()	Remove an item from the end of the list
getFirst()	Get the item at the beginning of the list
getLast()	Get the item at the end of the list

LinkedList Other Methods

descendingIterator() method which returns an Iterator object containing all elements of a LinkedList in the reverse order i.e from tail to head.

```
LinkedList<String> list = new LinkedList<String>();
Iterator<String> it = list.descendingIterator();
```

use **addAll()** method which takes Collection type as an argument to join two ArrayLists or a arraylist and linked list

```
LinkedList<String> linkedList = new LinkedList<String>();
 ArrayList<String> arrayList = new ArrayList<String>();
linkedList.addAll(arrayList);
```

removeFirstOccurrence(Object 0) to remove the first occurrence of a given element and use **removeLastOccurrence(Object 0)** to remove last occurrence of a given element in a LinkedList

```
list.removeLastOccurrence(”ONE");
```

Comparison b/w Array and Linked List

```
//accessing an element in an array

start=System.nanoTime();
Obj1.indexOf(4999);
elapsed = System.nanoTime()-start;
System.out.println("Elapsed (ArrayList):"+elapsed);

//output : Elapsed (ArrayList):2613807

start=System.nanoTime();
Obj2.indexOf(4999);
elapsed = System.nanoTime()-start;
System.out.println("Elapsed (LinkedList):"+elapsed);

//output : Elapsed (LinkedList):38035884

//Code demonstrates that arraylist is better for data access.
```

```
//removing an element in an array
//Obj 1 is an object of ArrayList
start=System.nanoTime();
Obj1.remove( index: 4999);
elapsed = System.nanoTime()-start;
System.out.println("Elapsed (ArrayList):"+elapsed);

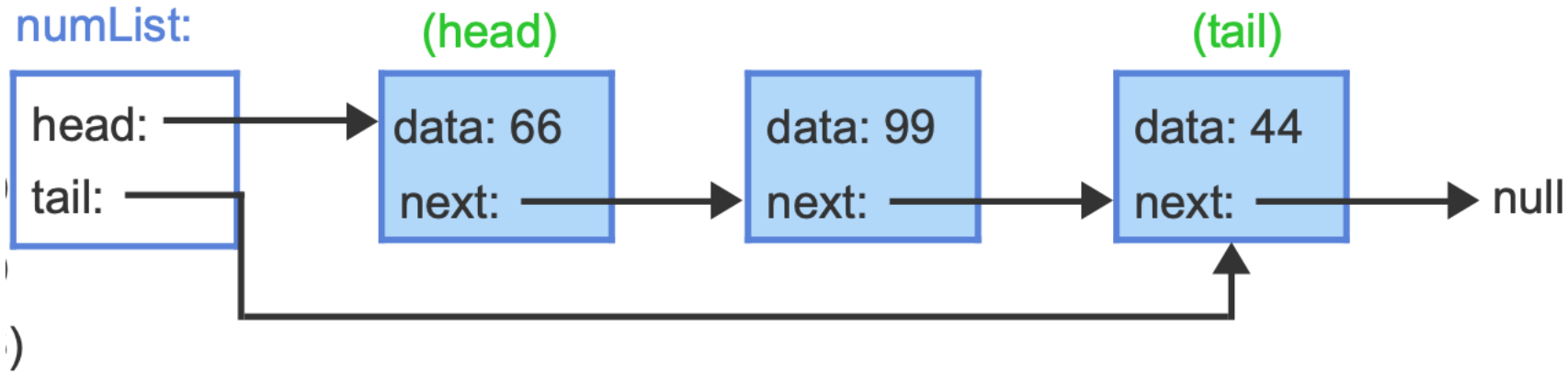
//output : Elapsed (ArrayList):2613807
//Obj 2 is an object of Linked List
start=System.nanoTime();
Obj2.remove( index: 4999);
elapsed = System.nanoTime()-start;
System.out.println("Elapsed (LinkedList):"+elapsed);

//output : Elapsed (LinkedList):38035884

//Code demonstrates Linked List is better for data manipulation.
```

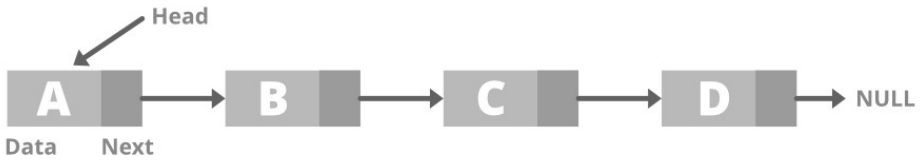
Linked List Terms

- **Node** – each node of a linked list can store a piece of **data**.
- **Next** – each node of a linked list contains *the next node* called **next**.
- **Head** – first node of the linked list.
- **Tail** – the last node of a linked list

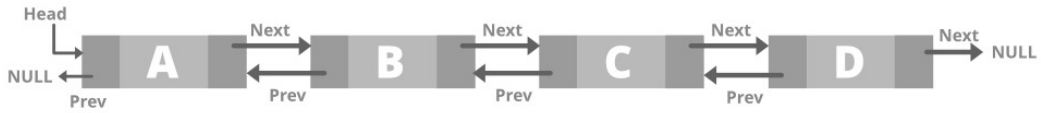


Types of linked list

Singly Linked List



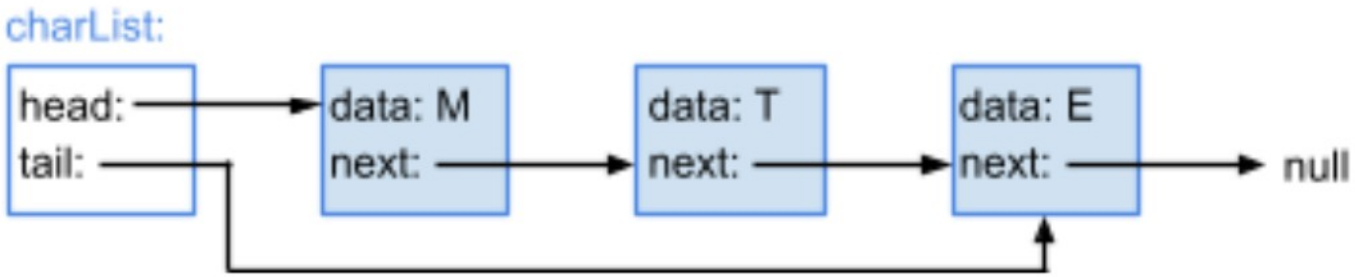
Doubly Linked List



Circular Linked List



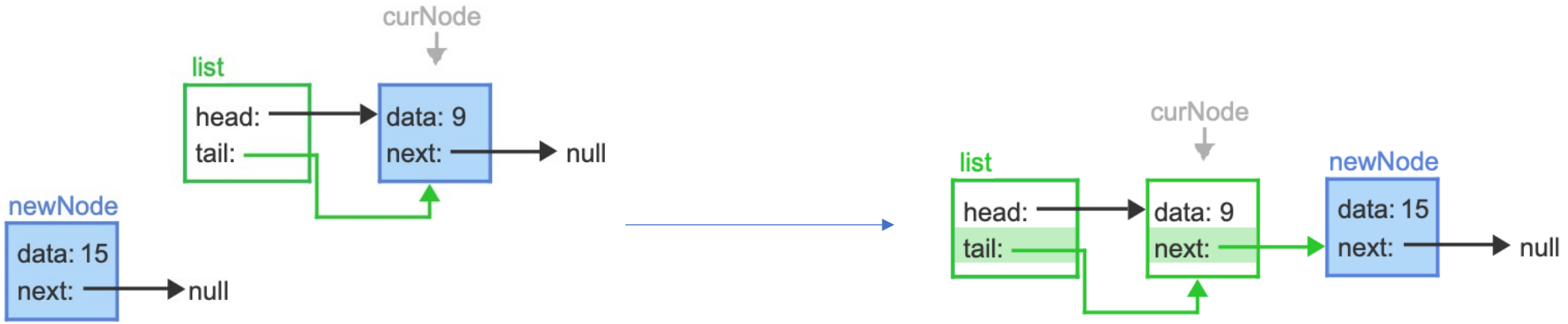
Singly Linked List



A ***singly-linked list*** is a data structure where each node has **data and a pointer** to the next node.

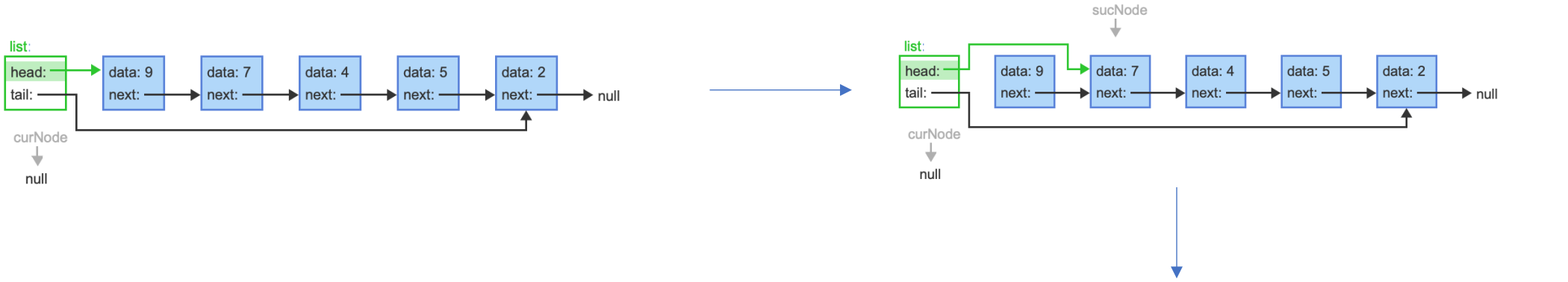
null is a special value indicating a pointer points to nothing.

Insert / Append / Prepend



- *Insert as list's first node:* If the list's head pointer is null, the algorithm points the list's head and tail pointers to the new node.
- *Insert after list's tail node:* If the list's head pointer is not null (list not empty) and curNode points to the list's tail node, the algorithm points the tail node's next pointer and the list's tail pointer to the new node.
- *Insert in middle of list:* If the list's head pointer is not null (list not empty) and curNode does not point to the list's tail node, the algorithm points the new node's next pointer to curNode's next node, and then points curNode's next pointer to the new node.

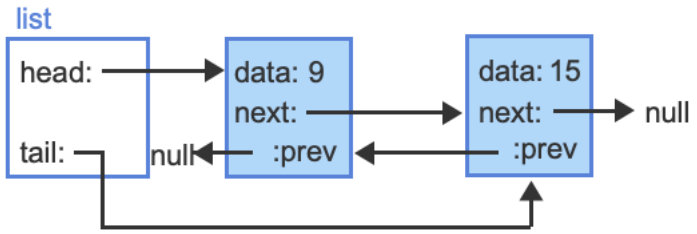
Delete



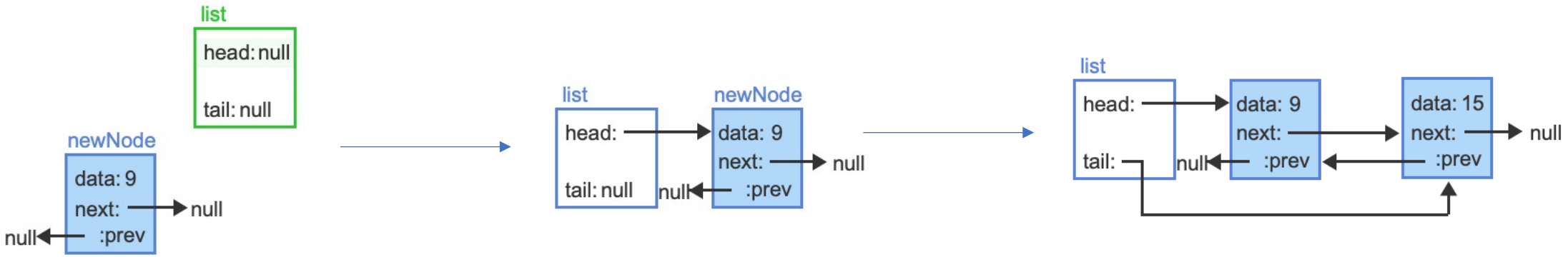
•*Remove list's head node (special case):* If curNode is null, the algorithm points sucNode to the head node's next node, and points the list's head pointer to sucNode. If sucNode is null, the only list node was removed, so the list's tail pointer is pointed to null (indicating the list is now empty).
•*Remove node after curNode:* If curNode's next pointer is not null (a node after curNode exists), the algorithm points sucNode to the node after curNode's next node. Then curNode's next pointer is pointed to sucNode. If sucNode is null, the list's tail node was removed, so the algorithm points the list's tail pointer to curNode (the new tail node).

Doubly Linked List

A ***doubly-linked list*** is a data structure for implementing a list ADT, where each node has data, a pointer to the next node, and a pointer to the previous node. The list structure typically points to the first node and the last node. The doubly-linked list's first node is called the head, and the last node the tail.



Doubly Linked List - APPEND



- *Append to empty list:* If the list's head pointer is null (empty), the algorithm points the list's head and tail pointers to the new node.
- *Append to non-empty list:* If the list's head pointer is not null (not empty), the algorithm points the tail node's next pointer to the new node, points the new node's previous pointer to the list's tail node, and points the list's tail pointer to the new node.